

MinimalAuth

Gavriluță Rareș-Andrei

¹ Facultatea de Informatică a Universității „Alexandru Ioan Cuza”, Iași
rares.gavriluta@info.uaic.ro

Abstract. MinimalAuth constă într-un caz de utilizare minimal pentru protocolul OAuth. Acesta permite unui client third-party să obțină acces limitat la fișierele resursă ale unui server proprietar prin orchestrarea unei interacțiuni de autorizare a clientului, ce evită astfel dezvăluirea parolei pe care utilizatorul o folosește în mod obișnuit.

Keywords: OAuth, token, security, fișiere-resursă, autorizare, client

1 Introducere

În modelul tradițional de autentificare client-server, clientul solicită o resursă cu acces restricționat (resursă protejată) aflată pe server prin autentificarea cu serverul folosind datele de logare ale utilizatorului. Prin urmare, pentru a oferi aplicațiilor terțe acces la resurse protejate, proprietarul resurselor este nevoit să-și împărtășească datele de autentificare cu aplicația terță în cauză. Acest lucru produce o serie de probleme și limitări:

- Aplicațiile terțe (clienții) sunt nevoite să stocheze datele de autentificare ale utilizatorului pentru utilizări ulterioare, de regulă o parolă în text clar.
- Serverele trebuie să accepte autentificarea prin parolă, în ciuda slăbiciunilor de securitate inerente parolelor.
- Aplicațiile terțe obțin un acces prea larg la resursele protejate, lăsând proprietarii fără nicio capacitate de a restricționa durata sau accesul la un subset limitat de resurse.
- Proprietarii de resurse nu pot revoca accesul unei terțe părți individuale fără a revoca accesul tuturor terților și trebuie să facă acest lucru prin schimbarea parolei terței părți.
- Compromiterea oricărei aplicații terțe compromise implicit parola utilizatorului și toate datele protejate de această parolă.

Protocolul MinimalAuth abordează aceste probleme prin introducerea unui nivel de autorizare și separarea rolului clientului de cel al resursei proprietar. În MinimalAuth, clientul solicită acces la date protejate care sunt găzduite de serverul de resurse și i se acordă un set de credențiale, diferite de cele folosite de utilizatorul-prorietar în mod obișnuit.

În locul folosirii datelor de autentificare ale proprietarului în vederea accesului la resurse protejate, clientul obține un token de acces – un șir ce denotă un scop specific, durata lui de viață și alte posibile atribute. Token-urile de acces sunt emise clienților terți de către un server de autorizare cu aprobarea proprietarului resursei. Clientul folosește mai departe acest token de acces pentru a accesa resursele protejate găzduite de serverul de resurse.

De exemplu, un utilizator (proprietar de resurse) poate autoriza un serviciu de printare (client) să acceseze poze protejate găzduite de un serviciu de partajat poze (server de resurse), fără a partaja username-ul și parola cu serviciul de printare. În schimb, utilizatorul se autentifică într-un server verificat și de încredere (server autorizare) pentru serviciul de printare, ca mai apoi acesta să emită serviciului de printare un set de credențiale (token acces).

2 Tehnologiile utilizate

2.1 Servere TCP

Atât pentru serverul de autorizare, cât și pentru serverul de resurse am implementat câte un server TCP în defavoarea unuia UDP din următoarele motive:

- TCP este orientat pe conexiune în sensul că, înainte ca două procese să transmită date, acestea trebuie “să facă cunoștință” între ele (un schimb preliminar de date ce stabilește parametrii cu privire la transferul care urmează). În cazul nostru, întreg protocolul de autorizare impune o conexiune directă între clienți și servere.
- TCP oferă garanția că datele nu sunt deteriorate, pierdute, duplicate sau livrate în neregulă între procesele care participă la transfer. Clienții noștri fac request-uri pentru a primi în schimb fișiere în masă, iar pentru acest tip de transfer TCP este cel recomandat.
- Modelul TCP/IP furnizează un mecanism de flow control, care înseamnă practic că TCP se va asigura că un expeditor nu copleșește un receptor trimițând pachete mai repede decât poate consuma. Având în vedere că protocolul nostru transferă fișiere (voluminoase) frecvent, nu ne-am dori ca buffer-ul de primire al clientului să fie supraîncărcat de serverul care transmite.

2.2 Arhitectura server TCP concurent – thread per client

Pentru serverul de autorizare la care clienții se vor conecta pentru a solicita un token de acces am ales ca arhitectura pe care se bazează să fie una de tip thread per client. O astfel de arhitectură este adecvată din două motive principale. În primul rând, dacă luăm în calcul că serverul nostru va fi solicitat frecvent, arhitectura alternativă (process per client) ce folosește `fork()`, nu este recomandată din cauza performanței (metoda `fork()` este relativ costisitoare de executat). În plus, la nivel de timp de execuție, implementarea cu threaduri este în general mai rapidă decât alte arhitecturi similare, inclusiv decât varianta optimizată “preforked – process per client”.

Tabel 1. Comparație între timpii de execuție al unor clienți TCP

TCP client	Timp de execuție (sec.)
Usual pattern (stop-and-wait)	...
Using select and blocking I/O operations	12.3
Using select and unblocking I/O operations	6.9
Using fork()	8.7
Using threads	8.5

2.3 Server TCP concurent prethreaded cu blocare mutex

Pentru implementarea serverului care găzduiește resurse (fișierele protejate) am introdus ca optimizare suplimentară, un server TCP concurent care deservește clienții printr-un mecanism de prethread-ing și care folosește de asemenea blocarea mutex de protecție a lui accept(). Față de serverul de autorizare, acesta creează un număr de threaduri ales de noi imediat ce este pornit, înainte să se conecteze clienții. Optimizarea provine din faptul că mecanismul prin care creăm în prealabil threaduri ce pot apela accept(), este mai rapid și mai simplu decât acela în care threadul principal apelează accept și tot el redirectează descriptorul celorlalte threaduri/procese copil. Mecanismul de blocare folosit (lacătul mutex) ne asigură că nu pot exista 2 threaduri care sa apeleze accept() simultan, scenariu pe care nu ni-l dorim.

Dat fiind faptul că putem specifica ca parametru numărul de threaduri ce vor fi create, acest lucru ne permite să prioritizăm sau nu performanța în defavoarea timpilor de așteptare, în caz că dorim acest lucru.

3 Arhitectura protocolului

3.1 Diagrama protocolului

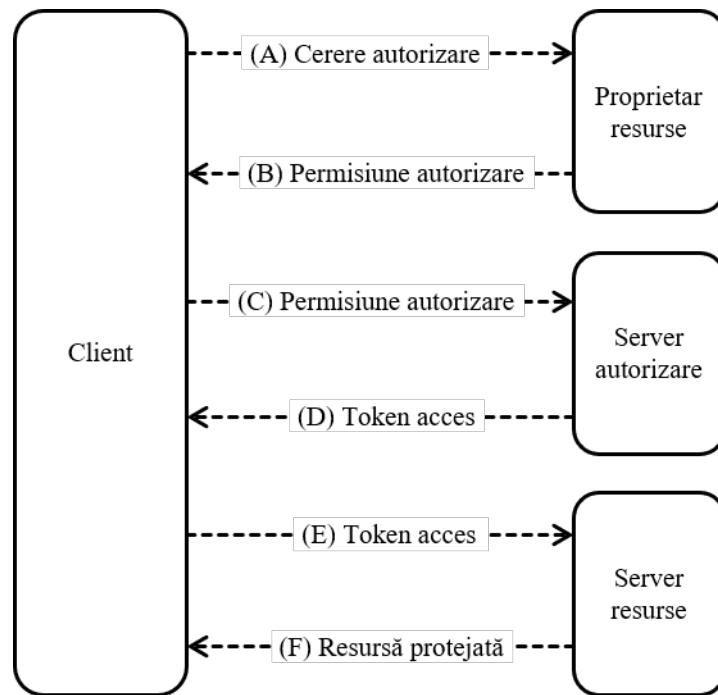


Fig. 1. Diagrama aplicației la nivel abstract

În figura 1 este ilustrată diagrama aplicației la nivel abstract și interacțiunile dintre principalii actori după cum urmează:

- (A) Clientul face o cerere de autorizare către proprietarul de resurse cu intenția de a accesa resursele protejate de acesta. În cazul nostru, ca metodă suplimentară de securitate, cererea nu se face direct, ci indirect, prin serverul de autorizare (procedeu ce va fi descris mai jos)
- (B) Clientul primește permișiunea de autorizare, ce poate fi exprimată în mai multe feluri. În cazul nostru permișiunea reprezintă un cod numeric de autorizare folosit mai departe.
- (C) Clientul solicită un token de acces de la serverul de autorizare, prezentând codul de autorizare primit mai devreme.
- (D) Dacă codul prezentat de client coincide cu cel emis, serverul de autorizare generează un token de acces, un număr de utilizări al acestuia, un token opțional de refresh, validează token-urile generate și le transmite clientului.
- (E) Clientul se conectează la serverul de resurse, prezintă token-ul de acces primit și solicită un fișier protejat

- (F) Serverul de resurse verifică dacă token-ul este valid și nu a expirat, iar în caz afirmativ, servește clientul cu resursa solicitată.

3.2 Conceptele implicate

Cod de autorizare. Codul de autorizare este modalitatea prin care proprietarul de resurse acordă permisiunea clientului de a solicita un token de acces. În locul solicitării permisiunii direct de la proprietar, clientul redirecționează proprietarul la serverul de autorizare, care generează un cod de unică folosință, iar acesta din urmă (s. de aut.) redirecționează înapoi proprietarul la client pentru a transmite acest cod. În acest pas, este necesar ca proprietarul să se autentifice sau să fie deja autentificat înainte să-i fie acordat codul de transmis clientului. Dat fiind faptul că singura autentificare se realizează între utilizatorul proprietar și serverul de autorizare, parola nu se dezvăluie niciunui client străin și rămâne protejată. În cazul de față vom presupune că utilizatorul este deja autentificat în serverul de autorizare. Acest mecanism aduce câteva beneficii de securitate importante printre care:

- garanția serverului de autorizare că, clientul care solicită token-ul este și cel încredințat de proprietar
- abilitatea s. de autorizare de a transmite token-ul de acces direct clientului, fără alți intermediari (inclusiv proprietarul) care ar putea să îl compromită.

Token de acces. Token-urile de acces sunt o formă de credențiale acordate pentru a permite accesul la resurse protejate. Acestea sunt de regulă generate sub forma unor șiruri, opace pentru client și care denotă mai multe atribute precum scopul, numărul de utilizări etc. (similar unui CNP). Aceste atribute pot fi trecute printr-un layer de abstractizare, astfel încât să fie înțelese doar de serverul de resurse.

Token de refresh. Token-urile de refresh sunt atribuite odată cu cele de acces și sunt destinate să fie folosite în momentul în care token-ul de acces expiră sau devine invalid. Utilitatea lor provine din faptul că odată autorizat, clientul nu mai este nevoit să parcurgă întreg protocolul pentru a obține un token de acces nou de la server, ci îl poate folosi pe acesta în locul codului de autorizare.

3.3 Acordarea codului de autorizare

Pentru a solicita un token de acces, clientul obține permisiunea de autorizare de la proprietarul de resurse. Această permisiune este exprimată printr-un cod de autorizare, folosit de client pentru a solicita token-urile.

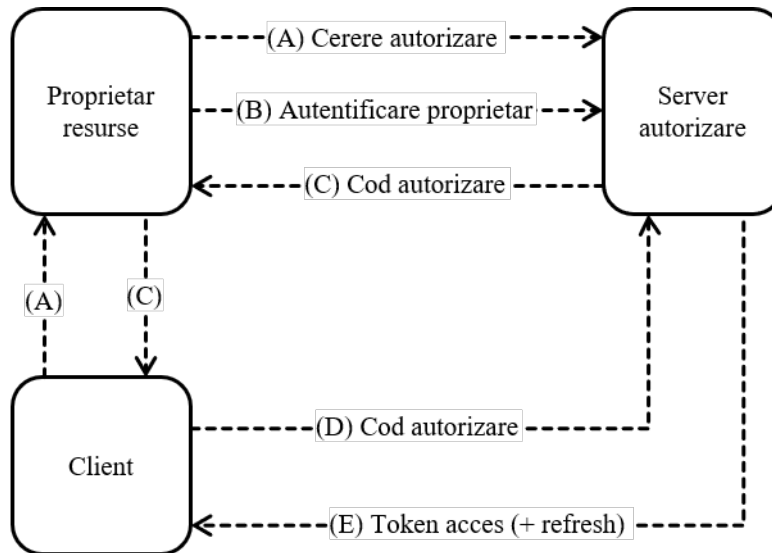


Fig. 2. Diagrama acordării codului de autorizare

Diagrama ilustrată în figura 2 cuprinde următorii pași:

- (A) Clientul cere permisiunea de a solicita un token redirectând utilizatorul către serverul de autorizare în vederea obținerii unui cod de autorizare
- (B) Serverul de autorizare se asigură că utilizatorul este autentificat și stabilește dacă proprietarul dorește într-adevăr autorizarea clientului respectiv
- (C) În caz afirmativ, serverul generează un cod de autorizare și redirecționează utilizatorul înapoi la client împreună cu acest cod numeric
- (D) Clientul preia codul mai departe și îl transmite serverului de autorizare în vederea obținerii unui token de acces.
- (E) Serverul de autorizare verifică codul primit și dacă coincide cu cel emis mai devreme, servește clientul cu token-ul cerut și opțional, cu unul de refresh.

4 Detalii de implementare

4.1 Cod relevant particular protocolului

Clientul redirecționează utilizatorul către auth_server. Transmite pid-ul către auth_server ca metodă de identificare

```

/* start */
printf("[client] Se autorizeaza client %d\n", pid);
printf ("Redirecting you to auth_server...\nIntrodu-
ceti codul primit de la [auth_server]\n");
fflush (stdout);

```

```

    /* ne conectam la server */
    if (connect (sd, (struct sockaddr *) &server, sizeof
(struct sockaddr)) == -1)
    {
        perror ("[client] Eroare la connect() auth_ser-
ver");
        return errno;
    }

    if (write (sd, &pid, sizeof(int)) <= 0)
    {
        perror ("[client] Eroare la write() pid spre
auth_server");
        return errno;
    }

```

Auth_server solicită confirmarea de la proprietarul de resurse. În caz afirmativ, generează un cod de autorizare și îl transmite utilizatorului. În caz contrar (codul primit ulterior nu coincide sau utilizatorul refuză autorizarea), auth_server va refuza clientul.

```

int grant_acces(void *arg)
{
    int pid; int auth_code, cl_code=0;
    char buf[100];

    struct thData tdL;
    tdL= *((struct thData*)arg);

    if (read (tdL.cl, &pid, sizeof(int)) < 0)
    {
        perror ("[client] Eroare la read() de la client");
        return errno;
    }

    printf("[thread %d] Clientul cu pid: %d solicita acces
la resurse\n Confirm? (y/n)\n", tdL.idThread, pid);
    gets(buf);
    if(strcmp(buf, "y")==0)
    {
        auth_code=rand()/1000000;
        printf("[thread %d] Tastati acest cod in client(%d) :
%d\n", tdL.idThread, pid, auth_code);
        read(tdL.cl, &cl_code, sizeof(int));
        if(auth_code==cl_code)
            return pid;
    }
}

```

8

```
        else
        {
            printf("Client refuzat. %d Nu coincide codul %d\n",
cl_code);
            return 0;
        }

    }
    else
    {
        printf("Client refuzat.\n");
        return 0;
    }
}
```

Serverul de autorizare validează clientul. Se generează token-urile, se validează și se transmit clientului

```
if((pid = grant_acces((struct thData*)arg))!=0)
{
    char * token=generate_token();
    char * refresh=generate_token();
    int nr_utilizari_token=generate_attempts();

    validate_token((struct thData*)arg, token, refresh,
nr_utilizari_token, pid);
    printf ("[thread %d] Token generat: %s - %d attempts
- refresh: %s\n",tdL.idThread, token, nr_utilizari_token,
refresh);
    raspunde((struct thData*)arg, token, refresh);
}
```

Serverul de resurse servește clienții care prezintă un token valid. Se verifică token-ul prezentat la fiecare request și dacă este valid, se decrementează nr. de utilizări și se transmite fișierul solicitat. În caz contrar, clientul va fi înștiințat că token-ul a expirat.

```
/* realizeaza primirea si retrimiteria unui mesaj unui
client */
int serveClient(int client, FILE* tokens, int i)
{
    char buffer[100];    /* mesajul */
    int bytes;           /* numarul de octeti cititi/scrisi */
    char token_read[13];
    char fileName[30];
```



```

    char msgrasp[100]=" ";          //mesaj de raspuns pentru
    client
    char line[40];

    bytes = read (client, buffer, sizeof (buffer));
    if (bytes < 0)
    {
        perror ("[thread] Eroare la read() de la cli-
ent.\n");
        return 0;
    }

    strncpy(token_read, buffer, 12);
    strcpy(fileName, buffer+13);
    printf ("[thread %d] Received: %s - File req: %s\n", i,
token_read, fileName);
    int found=0;
    while(fscanf(tokens, "%s", line) && !feof(tokens))
    {
        if(strstr(line, token_read)!=0 && atoi(line+13)>0)
        {
            printf("[thread %d] %s valid - Attempts rema-
ining: %d\n",i, token_read, atoi(line+13)-1);
            found=1;
            decrease_attempts(tokens, line);
            sendFile(client, fileName, i);
            break;
        }
    }
    if(found==0)
    {
        write(client, "Token expirat!", 14);
        rewind(tokens);
        return 0;
    }
    return 1;
}

```

Clientul face solicitări succesive la serverul de resurse. După primirea token-ului de acces, clientul va solicita de la `res_server`, în mod succesiv și la intervale de timp (în scop demonstrativ) câte un fișier ales aleator, până când expiră token-ul.

10

```
do
{
    const char * fileName = pick_file();
    sprintf(sendBuf, "%s %s", token, fileName);
    int len =strlen(sendBuf);
    sendBuf[len]='\0';
    printf("=====\n");
    printf("[client] Conectare la server resurse: %s\n",
sendBuf);
    if (write (sd_res, sendBuf, strlen(sendBuf)+1) <= 0)
    {
        perror ("[client] Eroare la write() spre server re-
surse\n");
        return errno;
    }

    if(read(sd_res, buff, 15) < 0)
    {
        perror ("[client] Eroare la read() de la res_ser-
ver");
        return errno;
    }
    else
    {
        printf("[client] %s\n",buff);
        if(strstr(buff, "Token expirat!")==0)
            receiveFile(fileName, sd_res);
    }
    sleep(5);
}
while(strstr(buff, "Token expirat!")==0);
```

4.2 Scenarii de utilizare

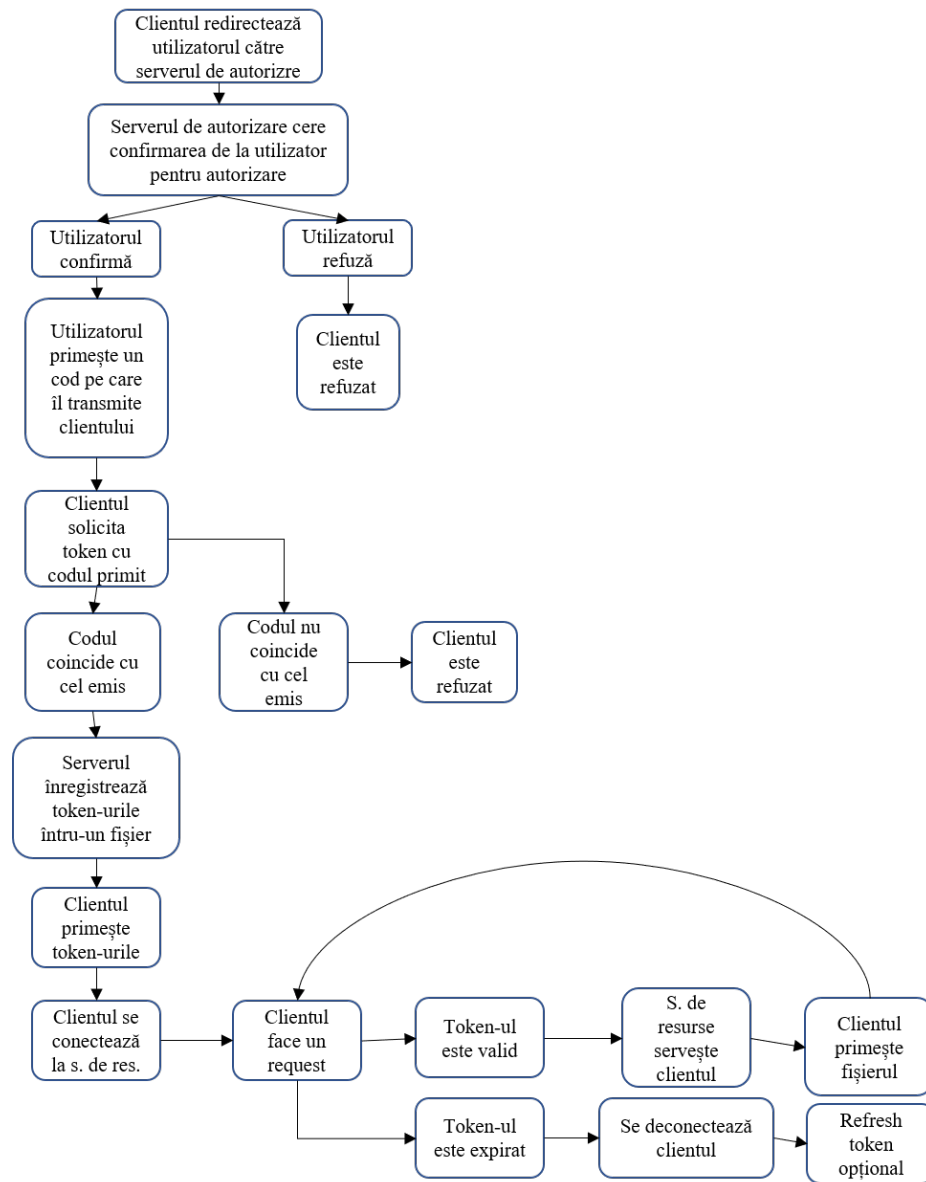


Fig. 3. Scenarii de utilizare

5 Concluzii

Protocolul mai sus prezentat rezolvă o problemă de securitate prezentă în autentificarea clasică prin aplicații terțe și poate fi implementat cu succes în foarte multe cazuri (aplicații web, aplicații desktop, aplicații bancare, smartphone-uri, device-uri smart în general etc.).

Totuși acesta poate fi îmbunătățit pentru a spori securitatea și performanța. Una din soluții am implementat-o parțial și anume, token-urile de refresh. Acestea presupun că odată autorizat, clientul nu mai este nevoit să parcurgă întreg protocolul pentru a obține un token de acces nou de la server, ci îl poate folosi pe acesta în locul codului de autorizare. Acest mecanism ne poate optimiza de exemplu un scenariu în care dorim să acordăm clientului un token cu un anumit nr. de încercări, dar doar într-un interval fix de timp.

O altă posibilă îmbunătățire se mai poate face la nivelul codului de autorizare. Extensia "Proof Key for Code Exchange" (PKCE) descrie o tehnică pentru clienți de a minimiza pericolul interceptării codului de autorizare. Tehnica presupune ca clientul să creeze mai întâi un cod secret și apoi să utilizeze din nou acel cod secret atunci când schimbă codul de autorizare pentru un token de acces. În acest fel, dacă codul este interceptat, nu va fi util deoarece cererea de token se bazează pe codul secret inițial.

6 Bibliografie

1. <https://datatracker.ietf.org/doc/html/rfc6749>
2. <https://en.wikipedia.org/wiki/OAuth>
3. <https://developer.okta.com/blog/2017/06/21/what-the-heck-is-oauth>
4. <https://www.udemy.com/course/oauth-2-simplified/>